Timely Real-Time Intervention and Guidance System for Supporting Students in Labs

Abubakar Othman (F221611)

MSci Computer Science and Mathematics

Department of Computer Science

Supervisor: Dr Firat Batmaz

Module Code: 25COD290

Final Deliverable 2026

Abstract

Contents

1 Introduction **7**

1.1 Problem . . . 7

1.2 Proposed Solution . . . 7

1.3 Aims and Objectives . . . 8

1.4 Risks and Mitigation . . . 8

1.5 Project Approach . . . 8

2 Background and Requirements **10**

2.1 Learning Analytics and Dashboards . . . 10

2.1.1 Learning Analytics in Higher Education . . . 10

2.1.2 Dashboards for Learning Analytics . . . 10

2.1.3 Real-Time Data Analytics and Visualisation . . . 12

2.2 Modelling Student Behaviour . . . 13

2.2.1 Modelling Student Struggle . . . 13

2.2.2 Knowledge Tracing and Bayesian Student Models . . . 14

2.2.3 Modelling Task Difficulty . . . 15

2.3 Data-Driven Personalisation . . . 16

2.3.1 Collaborative Filtering . . . 16

2.3.2 Text Mining and Mistake Pattern Recovery . . . 17

2.4 Retrieval-Augmented Feedback . . . 17

2.4.1 Retrieval-Augmented Generation . . . 17

2.4.2 Dense Retrieval and Approximate Nearest Neighbour Search . . . 18

2.5 Existing Systems . . . 18

2.5.1 Learning Management System . . . 18

2.5.2 Instructor Facing Dashboards . . . 19

2.5.3 Early Warning and At-Risk Student Systems . . . 21

2.6 Research Gaps . . . 22

2.7 Requirements Specification . . . 22

2.7.1 Functional Requirements . . . 22

2.7.2 Non-Functional Requirements . . . 23

2.7.3 Requirements Prioritisation (MoSCoW) 24

3 Design and Modelling **25**

3.1 System Architecture 25

3.2 Data Endpoint 26

3.3 Baseline Analytics Design 27

3.3.1 Student Struggle Model 28

3.3.2 Temporal Smoothing 29

3.3.3 Question Difficulty Model 29

3.3.4 Alternative Modelling Approach: Collaborative Filtering 31

3.3.5 Mistake Clustering 33

3.4 Advanced Model Design 33

3.4.1 Measurement Confidence 33

3.4.2 Item Response Theory 33

3.4.3 Bayesian Knowledge Training 33

3.4.4 Improved Struggle Model 33

3.5 Retrieval-Augmented Generation Feedback Design 33

3.6 Interaction and UI Designs 33

3.6.1 Design Principles 33

3.6.2 Instructor Views 34

3.6.3 Lab Assistant View 36

3.6.4 Visual Encoding of Metrics 36

4 Implementation **37**

4.1 Implementation Overview 37

4.2 Technology Stack and Software Selection 38

4.3 System Structure 38

4.3.1 Instructor System 38

4.3.2 Assistant System 38

4.3.3 Shared Runtime State 38

4.4 Data Pipeline 38

4.4.1 Endpoint Retrieval and Parsing 41

4.4.2 Data Normalisation and Structuring . . . 41
4.5 Session Management . . . 41
4.5.1 Live Session Lifecycle . . . 41
4.5.2 Saved Session History and Restoration . . . 41
4.6 Analytics Implementation . . . 41
4.6.1 Incorrectness Scoring . . . 41
4.6.2 Baseline Student Struggle Model . . . 41
4.6.3 Baseline Question Difficulty Model . . . 41
4.6.4 Collaborative Filtering . . . 41
4.6.5 Mistake Clustering . . . 41
4.7 RAG Suggested Feedback . . . 41
4.7.1 Hybrid Two-Layer Architecture . . . 41
4.7.2 Embedding Model (all-MiniLM-L6-v2) . . . 41
4.7.3 Generation and Prompting . . . 41
4.7.4 Suggestion Caching . . . 41
4.7.5 Graceful Degradation and Privacy . . . 41
4.8 Advanced Model Implementation . . . 41
4.8.1 Measurement Confidence . . . 41
4.8.2 Item Response Theory (IRT) Difficulty . . . 41
4.8.3 Bayesian Knowledge Training (BKT) Mastery . . . 41
4.8.4 Improved Struggle Model . . . 41
4.9 Lab Instructor System . . . 41
4.9.1 In Class View . . . 41
4.9.2 Student Detail View . . . 41
4.9.3 Question Detail View . . . 41
4.9.4 Data Analysis View . . . 41
4.9.5 Comparison View . . . 41
4.9.6 Settings View . . . 41
4.9.7 Previous Sessions View . . . 41
4.10 Lab Assistant System . . . 41
4.10.1 Session Join Flow . . . 41

4.10.2 Waiting and Assignment States . . . 41
4.10.3 Live Assistant Allocation . . . 41
4.11 Problems Encountered and Solutions . . . 41

5 Evaluation **42**
5.1 Evaluation Design . . . 42
5.1.1 Scope of Evaluation . . . 42
5.1.2 Evidence Sources . . . 42
5.1.3 Retrospective Evaluation Protocol . . . 42
5.1.4 Limitations of Evaluation . . . 42
5.2 Functional Testing . . . 42
5.2.1 Data Ingestion and Session Handling . . . 42
5.2.2 Analytics and Model Behaviour . . . 42
5.2.3 Instructor Dashboard Features . . . 42
5.2.4 Lab Assistant Workflow . . . 42
5.3 Non-Functional Testing . . . 42
5.3.1 Performance and Refresh Behaviour . . . 42
5.3.2 Usability and Interpretability . . . 42
5.3.3 Robustness and Failure Handling . . . 42
5.4 Results . . . 42
5.4.1 Baseline System Results . . . 42
5.4.2 Advanced Model Comparison . . . 42
5.4.3 Observed Strengths and Weaknesses . . . 42
5.5 Discussion . . . 42
5.5.1 Model Disagreement . . . 42
5.5.2 Limitations . . . 42

6 Conclusion **43**
6.1 Summary of the Project . . . 43
6.2 Key Findings Contributions . . . 43
6.3 Future Work . . . 43

A Code Snippets **44**

B User Interface Screenshots **45**

C Detailed Test Results **46**

D Themes and References **47**

E Formulae **48**

F Formulae Derivation **49**

References **50**

1 Introduction

1.1 Problem

Computer Labs are fast paced environments where many students are working simultaneously on different questions are their own pace. Some students complete questions slower than others, and some may have prior knowledge. In practice, it is difficult to provide timely, high-quality support to students, monitor their progress, and prioritise those who are struggling most so they do not fall behind. We aim to address these issues in this paper.

Our current computer labs are highly dependent on staff surveying the room or counting on students asking for help. The current approach to computer labs has a range of limitations. Students who hesitate to ask for help in a lab environment may not receive the support they need. A student can be struggling significantly, but we may have no indicator of this struggle. Additionally, knowing what students require more help is almost impossible with the current approach, even with staff surveying the lab it is hard to quickly judge who needs immediate help. Furthermore, there is no structure to the allocation of lab assistants, without any indicator of who is struggling and allocating students smartly we do not provide help in the most efficient way possible.

These limitations become more severe the more class size increases. With more students, it becomes harder to identify struggling students and prioritise them for help.

The central problem is addressed in this project is therefore how to provide timely, high-quality feedback to students in computer labs, using the data generated from student work. The goal is not to replace human teaching but to enhance it.

1.2 Proposed Solution

In this project, we had addressed these issues and limitations with a real-time learning analytics dashboard we developed for use in computer lab sessions. The system supports lab teachers and assistants by transforming live student data into interpretable metrics we can use quickly during labs.

The dashboard integrates data from an existing endpoint and processes it into a set of practical metrics. The include measures derived from attempt number, time spent and average numbers of attempts to complete a question. From these signals we compute indicators of student struggle and question difficulty, allowing staff to identify which students are struggling and which questions are causing the most difficulty.

The system aims to assist rather than replace human judgement. The dashboard does not by default make its own decisions. Instead it provides a clearer operational view of what is happening in the lab so instructors can allocate resources more effectively. The system also allows for lab instructors to allocate lab assistants within the session, allowing them to be allocated through the system rather than informally.

The project has smart device integration, sending who to help to lab assistant's phones with recommendations on how to help students.

1.3 Aims and Objectives

The primary aim of this project was to design and implement a system that enables lab teachers and lab assistants to make more well-informed decisions during computer lab sessions.

The secondary aim is to extend our system to smart devices. Whilst the core system is a desktop-based dashboard, we can improve usability by sending notifications or messages to smart devices to enable easier communication with assistants.

The aims will be addressed through the following objectives:

1. **Identify relevant literature and existing systems** related to learning analytics, real-time data visualisation, and AI in education, to identify existing limitations in current approaches to relevant areas
2. **Formalise system requirements** by defining functional and non-functional requirements for the real-time computer lab support system
3. **Design Data Model** that extracts relevant data provided into meaningful metrics suitable for real-time analysis
4. **Design system architecture** to support live data ingestion, prioritisation and presentation while remaining extensible to smart devices
5. **Develop a struggle and difficulty model** so that we can take the relevant values to produce metrics which will be used in showing the relevant information
6. **Implement live data analytics dashboard** for real-time data visualisation, prioritisation and intuitive interactivity
7. **Evaluate approach** through functional and non-functional aspects to determine the feasibility and usefulness in a computer lab session

Together, these objectives guided the project toward becoming a full-fledged EdTech application that aimed to improve learning outcomes by enabling more efficient and targeted support.

1.4 Risks and Mitigation

The project involved developing a real-time data analytics system for use during computer lab sessions. As such, potential technical and methodological risks were identified and addressed through deliberate design decisions and extensive testing.

1.5 Project Approach

The project followed an incremental and implementation-focused approach. Early work focused on understanding the educational and technical context through a literature review and analysing the dataset. This established the problem framing, clarified the type of analytics that would be most

Risk Description	Potential Impact	Mitigation Measures
Latency in real-time data processing	Delays in updating the dashboard can reduce the usefulness and impact of timeliness of support	Ensure there are lightweight analytics and incremental updates
Unstable struggle and difficulty estimates due to insufficient data	Irregular interactions may lead to incorrectly identifying struggling students	Establish a baseline for suitable data or use an LLM to identify whether data is suitable
Overly complicated dashboard	Overcomplicating the dashboard may be more distracting and be unintuitive	Seek constant feedback on the UI so that we can implement it correctly and focus on main metrics
Unable to implement smart glasses functionality	Due to technical constraints, we may not be able to fully integrate the dashboard to smart glasses	Explore smartwatch solutions or simplify what goes into the smart glasses
Limited targeted towards different modules	A model that works well on one model may not work well on another	Design models over generic interaction patterns rather than specific module indicators

Table 1: Identified Risks and Mitigation Strategies

useful to the lab instructor during a computer lab and also addressed the limitations of current approaches.

The next stage focused on system design. This included defining the system architecture, understanding the structure of the data endpoint and deciding which data can aid in creating our student struggle and question difficulty metrics. The process also also considered how the system could be extended to smart devices, ensuring that the design was flexible enough to accommodate future extensions.

Implementation then progressed from core functionality to increasing the complexity of our analytics. The priority was to establish reliable data retrieval, parsing and processing. Once this was in place, we extended the dashboard to include live monitoring, drill down analysis, model comparison, historical sessions and assistant coordination. More advanced analytical components were implemented such as collaborative filtering, mistake clustering and alternative modelling approaches which use Item Response Theory and Bayesian Knowledge Tracing.

The incremental approach was appropriate for this project as it reduced risk and made it possible to validate each layer of the system before moving on to the next step. This was approach allowed us to turn a functioning pipeline into a more complete analytics platform.

Overall, the project aligned with the practical aim of the dissertation, which was to produce a working system that improves awareness and prioritisation in live computer labs.

2 Background and Requirements

2.1 Learning Analytics and Dashboards

2.1.1 Learning Analytics in Higher Education

In 2012, the use of data analytics was shown to improve student success by providing real-time feedback, demonstrating the impact of data informed intervention in higher education [1]. Learning analytics is the measurement, collection, analysis, and reporting of data about learners and their contexts in order to understand and optimise learning and the environment in which it occurs [1–4]. The growth of online education since the 1990s has contributed significantly to the advancement of learning analytics, as student data can now be captured and analysed enabling educators to make more data-driven decisions [1, 5, 6]. Learning analytics allows educators to make data-driven decisions.

Learning analytics leverages data sources; many types of data sources have been explored and used to help educators. Relevant data sources include Massive Open Online Courses (MOOCs), clickstream data (records of user interactions, such as clicks and time spent), and course content [5]. There are also data sources that, in the context of our project, we can get away without using, such as demographic data and gaze patterns. Whilst 35% of the studies used more than one data source, the majority of studies still used only one data source. It is argued that the use of multiple data types will contribute more effectively to learning outcomes [5].

The outputs of learning analytics have proven to be valuable for pedagogical decision-making. Use includes personalisation and feedback, which can tailor recommendations to students' behavioural profiles to promote diverse learning [5]. Another use case is the VITAL project, which confirmed that flipped classroom models were effective, as students accessed materials before attending face-to-face sessions [7]. However, several studies note challenges and limitations in learning analytics. These include how analytics leverage observable behaviours such as clicks and time online rather than cognitive processes [8]. Learning analytics also raises ethical concerns, such as privacy, when moving solutions out of labs [8, 9]. Many restrictions must be determined, as protecting students' identities and data is a priority, which we shouldn't lose sight of when trying to improve learning outcomes.

With the current utilisation of learning analytics, there is yet to be an approach that leverages smart devices in computer labs, in particular, smart watches and smart glasses. By leveraging smart devices, we can significantly increase the efficiency of lab assistance. From a centralised point, we can allow the lab teacher to send instructions to lab assistants based on these analytics, such as which questions are hardest and which students need the most help, thereby improving learning outcomes [5, 8, 9].

2.1.2 Dashboards for Learning Analytics

Learning analytics have been leveraged to use dashboards as valuable tools for both educators and learners, such as making educators aware of subtle interactions in their courses and allowing learners to gain insight into their learning actions and the effects they may have [10, 11]. Because of capabilities such as these, dashboards are the primary representation of learning analytics, as

they can help improve learning outcomes and make the most of the data generated by learning analytics [10, 11].

SAM was an early dashboard developed on data from learning environments, with both teachers and learners as target user groups [12]. SAM allowed teachers to separate students who were doing well from those at risk based on time spent and resource-use metrics. Teachers gain insight into the time learners spent on tasks and can compare with their estimates, whilst resource use provides insights on how and how often students used documents and tools, it also allows discovery of resources external to the course [12]. Metrics could also be added, depending on data detail, to increase understanding of specific tasks [12]. EMODA is a more recent approach to the learning dashboard; it takes into account emotion, which researchers have shown to play a key role in learning, regulation processes, and strong coupling to motivation and achievement [11]. EMODA enables teachers to monitor learners' emotions during online learning sessions by analysing audio, video, self-reports, and interaction traces [11].

Edsight is a web-based learning analytics dashboard developed with responsive web design that accommodates different device types and is accessible to educators [13]. When first logging in, teachers are presented with all the practical measure data they have collected thus far, the classrooms with the most data, and how many days have elapsed since they last logged data for analysis [13].

Figure 1: "Overview," "Report," and "Activity List" modes, displayed in desktop, tablet, and phone screens.

The report is the main dashboard in Edsight, where teachers can select filters such as date and classroom for display. Once a filter is applied, different results are displayed.

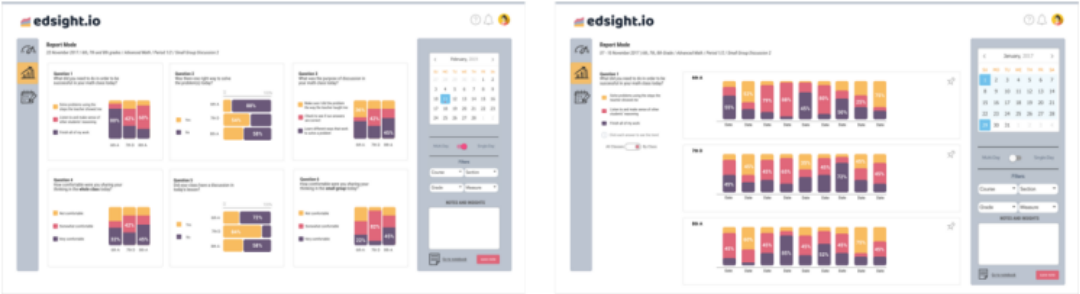

Figure 2: (L) A single-event report displayed in Edsight, with filters on the right side. (R) Multi-Day view, with the ability to compare patterns across classes and sections.

Most current approaches to the learning dashboard do not live in lab sessions, as there is more course-level monitoring than session-level monitoring [10, 11, 13]. This also has many benefits, helping students on the spot and allowing them to get the most out of their computer labs [13].

2.1.3 Real-Time Data Analytics and Visualisation

Real-time analytics refers to the practice of collecting and analysing data as it is generated, with minimal latency between data collection and analysis [14]. When we can process data as it comes in, we can gain insight and understand what is happening within a relevant context almost instantly. Such a practice is used in applications where data timeliness is critical, such as smart pricing and predictive maintenance [14]. Traditional data analysis, where data is captured and stored, and insights are pushed out of storage. In contrast, real-time data analysis allows data to be analysed in real time. Real-time data analytics have proven to be highly valuable, as evidenced by the success of Netflix, which has perfected the art of predicting its customers' next favourite show [14].

Real-time analytics has emerged as a pivotal tool across multiple sectors, including retail, revolutionising decision-making processes and operational strategies [15]. Retail giants such as Walmart and Amazon have leveraged real-time data analytics to personalise marketing campaigns, optimise pricing strategy, and efficiently manage inventory [15]. Technologies such as RFID, IoT, and advanced analytics platforms were used to gather real-time data, analyse customer behaviour, and make data-driven decisions [15]. A less utilised but more relevant application is in education, for example, a real-time machine learning application that detects learners' attention levels and participation using sensory data. The data allows teachers to recalibrate their teaching as necessary [16].

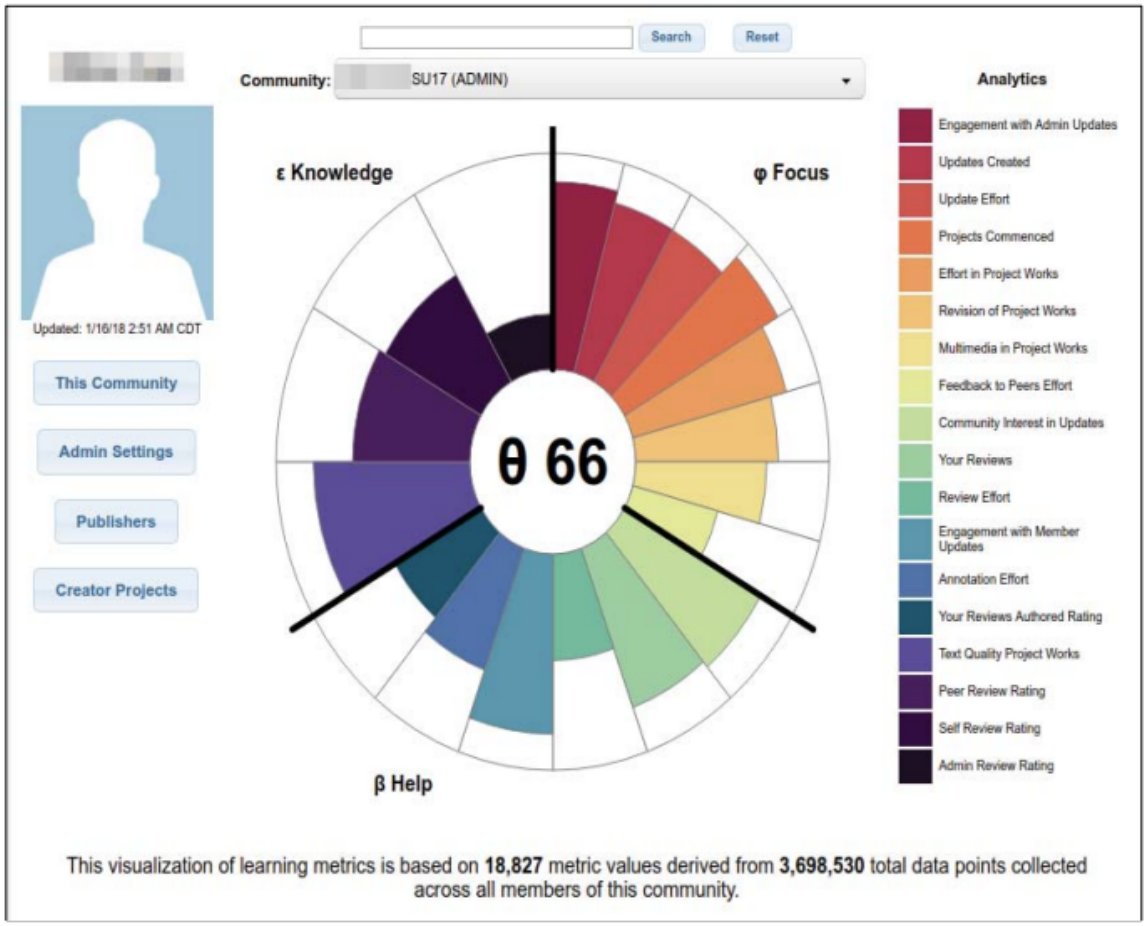

Figure 3: The Learning Analytics Interface for a dashboard which collects data based on student attention

Applying real-time analytics in computer labs will prove to be very beneficial for students and teachers alike. By collecting and processing data as it comes in, we will be able to update dashboards in real time and provide teachers with the most up-to-date information, enabling relevant, timely support. This would be better than a traditional data analysis approach, in which we review the data after the lab has finished, because our findings may not be relevant by then [15, 16].

2.2 Modelling Student Behaviour

2.2.1 Modelling Student Struggle

A key challenge in supporting students during lab sessions is identifying which students are struggling and may require timely intervention. Studies in education have approached struggle detection at

different temporal scales, ranging from within-assignment detection to course-level. Dong et al. highlighted an approach that detects moments of struggle during a session based on time spent and progress relative to peers, achieving over 77% accuracy when validated against expert judgement on whether intervention was needed [17]. Struggle has been operationalised as failing the same unit test consecutively for more than four submission events. providing a clear threshold-based criterion [17].

Beyond threshold-based approaches, more sophisticated methods have been explored. Or examined novice programmers' testing behaviour as an early step toward defining coding struggle, identifying patterns such as repeatedly submitting the incorrect answer for more than four submission events, signalling that students have stopped making meaningful progress [18]. To move this even further, Estey et al. proposed a trajectory metric that measures changes in programming behaviour over time across a four-semester study of 514 students [19]. Their early predictor identified 81% of students who struggled throughout the course, but also flagged 43% who went on to succeed, highlighting the difficulty of distinguishing slight from serious struggle. Their trajectory metric reduced this false-positive rate to 11%, demonstrating the importance of tracking how performance changes rather than measuring the students' current level [19].

Another approach uses machine learning to build a graphical model of how students navigate through programming assignments. By capturing the full solution trajectory, Piech et al. showed that a student's progression path is predictive of later difficulty, motivating the inclusion of the student's trajectory to track changes in answer quality across submissions [20].

A complementary approach is knowledge tracing which models how a student's mastery of underlying skills progresses over time and it is examined in the next subsection.

Recent work on Large Language Models (LLMs) is also relevant as AI-generated feedback can provide more valuable insights than raw correctness alone. Koutcheme et al. show that language models LLMs that are better at helping repair student programs also provide better explanations of coding issues, suggesting that the quality of LLM feedback can be studied empirically rather than assumed [21] More broadly, Pitts et al. survey LLM-based applications in programming education and argue that these systems are most effective when combined with human oversight rather than fully autonomous replacements [22]

Together, these works demonstrate different methods for evaluating struggle. These metrics are best captured through a combination of temporal, correctness-based and behavioural signals.

2.2.2 Knowledge Tracing and Bayesian Student Models

A related but distinct line of work is knowledge tracing, which models how a student's mastery of underlying skills progresses over time. Bayesian Knowledge Tracing (BKT) remains attractive because it is interpretable and computationally manageable. Khajah shows that BKT can gain a performance boost by merging it with Item Response Theory (IRT) to model multidimensional problem difficulties and student abilities, without fitting student specific parameters, allowing the model to be generalisable to new students [23]. In programming education, Kim et al. propose a knowledge tracing approach that integrates students' questions and automatically extracted skill information, showing that textual interaction data can provide useful signals about understanding misconception patterns [24]. This is relevant to the current project, as it suggests that struggles

during computer labs should not be treated solely as repeated failures but also as informed by temporal behaviour, how the student develops, and question-specific context.

Bayesian Knowledge Tracing was formalised by Corbett and Anderson in the context of an intelligent programming tutor, which treats the student's knowledge of each skill as either in a learned state or an unlearned state that can transition from unlearned to learned based on the student's interactions [25]. The model is governed by four interpretable parameters: an initial-learning probability, $p(L_0)$, an acquisition probability, $p(T)$ that a skill transitions from unlearned to learned at each practice opportunity, a guess probability, $p(G)$ for correct responses without mastery and a slip probability, $p(S)$ for incorrect responses despite mastery. The probability of a correct response on the next opportunity is then

$$p(C) = p(L) \cdot (1 - p(S)) + (1 - p(L)) \cdot p(G), \tag{1}$$

where $p(L)$ is the current estimate of mastery [25]. The original formulation assumes no forgetting, so the latent state cannot transition from learned back to unlearned.

Yudelson et al. extended BKT by individualising parameters across students and showed across KDD datasets that this yielded tangible improvements when predicting unseen student data [26]. They found that individualising the speed of learning $p(T)$ produced consistent gains in predictive accuracy, while individualising the prior $p(L_0)$ alone had no measurable effect, suggesting learning rate is the more important axis of student variation for BKT to exploit.

More recent work has explored neural alternatives to BKT such as deep learning approaches. Piech et al. proposed Deep Knowledge Tracing (DKT), with two different types of Recurrent Neural Networks (RNNs): a Long Short Term Memory (LSTM) based model and a vanilla RNN model with sigmoid unit. These models predict a student's future performance based on their past activity. This resulted in a 25% gain in the area under the curve (AUC) over the best previous result on a knowledge tracing benchmark as well as showing that knowledge tracing models do not need expert annotations [27]. However, Khajah et al. subsequently showed that BKT extended with forgetting, latent student abilities, and automatic skill discovery matches DKT performance on average across four different data sets, arguing that the gains of deep approaches stem from added flexibility rather than from any representation discovery [28]. For an instructor facing dashboard, where the interpretability of mastery estimates is essential, BKT remains a defensible choice.

2.2.3 Modelling Task Difficulty

Alongside students' struggles, it is important to identify which questions pose the greatest difficulty for the class as a whole so that teachers can address the problem in front of the whole class rather than just for the student. Dannath et al. examined task-level struggle detection methods in intelligent tutoring systems for programming, noting that prior work focused on dropout prevention rather than understanding the student's path towards the correct solution for a specific task [29]. The separation in modelling is important as students may appear to be struggling with a genuinely difficult question rather than with individual performance. Additionally, if most of the class is struggling with a problem, it will be faster to go through the question with the whole class.

Task difficulty can be inferred from indicators such as failure rate, average time spent, number of attempts and the amount of feedback used before reaching a correct answer [20,29]. These measures are useful as they are directly observable and can be updated during a live session with low cost,

but they do not separate question difficulty from variation in student ability.

Alternatively, we can model difficulty using Item Response Theory (IRT), which offers a more principled approach by accounting for how difficult a question actually is, rather than raw counts, as well as a student's latent ability [30, 31]. Baucks et al. uses IRT to analyse course difficulty variation and show how applying latent variable models can uncover meaning patterns in course rigor that is hidden when relying on simple metrics [32]. In online learning environments that allow multiple attempts, Pankiewicz argues that measuring task difficulty becomes more complex because task difficulty and feedback shape how it should be interpreted, motivating methods that go beyond raw metrics. [33] These works support the view that question difficulty should be modelled separately from student struggle and, where possible, that composite measures should be used rather than a single measure.

Overall, the literature suggests that task difficulty should be deduced from a combination of correctness, time, attempts, and progression-related information rather than from a single descriptive statistic [20, 29].

2.3 Data-Driven Personalisation

2.3.1 Collaborative Filtering

Beyond parametric approaches discussed in the previous subsection, more sophisticated data-driven methods have been explored in the educational recommender systems literature. Collaborative filtering is an alternative method used for personalising user experiences [34]. Unlike methods that rely on parameters, Collaborative filtering leverages the collective behaviour of users with similar histories to create preferences, on the principle that users with similar histories share similar needs. This method has been applied primarily in educational contexts to provide more efficient and accurate course suggestions than traditional methods. This can be done with different types of models, such as linear regression and factor decomposers. It is evident that integrating intelligent recommendation technology significantly enhances resource utilisation and the overall student learning experience

A user-based collaborative filtering method identifies a set of k nearest neighbours for a given student based on a similarity measure over their profiles [35]. The behaviour of the neighbours are aggregated to produce a prediction. The approach is shown to be effective in sparse datasets, where explicit ratings are unavailable and implicit signals such as time-on-task and submission frequency must serve as proxies for preference. A known limitation is that when a student has little interaction history, similarity cannot be computed reliably, leading to unstable early-session predictions.

In the context of lab struggle detection, collaborative filtering offers a strong alternative to our scoring approach. Rather than computing a fixed weighted score for each student, it can identify students whose interaction patterns closely resemble peers who need help, inferring that they too may need help. However applying collaborative filtering to a live computer lab comes with its constraints as students who may need help is limited and the start of the session will have very limited data. These constraints backs the use of a scoring approach, whilst the collaborative filtering approach is a strong alternative.

2.3.2 Text Mining and Mistake Pattern Recovery

Where student responses are free-text rather than numerical or categorical, identifying common mistake patterns requires text-mining techniques that combine information retrieval and unsupervised clustering. Salton et al. introduced the vector space model for information retrieval, which represents documents and queries as vectors in a vector space, allowing for the computation of similarity between them [36]. The standard weighting scheme is term frequency-inverse document frequency (TF-IDF) formalised in modern information retrieval as

$$\mathrm{tf} - \mathrm{idf}_{t,d} = \mathrm{tf}_{t,d} \cdot \log \frac{N}{\mathrm{df}_t}, \tag{2}$$

where $\mathrm{tf}_{t,d}$ is the frequency of term t in document d, N is the total number of documents, and df_t is the number of documents containing term t [37].

Once each document is represented as a TF-IDF vector, similar documents can be partitioned into clusters that share characteristic content. MacQueen's k-means algorithm partitions n points into k clusters to minimise within cluster variance [38]. Formally, this corresponds to choosing k centers C that minimise the sum of squared distances from each point to its closest center,

$$\phi = \sum_{x \in X} \min_{c \in C} \|x - c\|^2, \tag{3}$$

where X is the set of data points [39]. Because this objective is non-convex and sensitive to initialisation, Arthur and Vassilvitskii's k-means++ seeds initial centers by selecting each new center with probability proportional to its squared distance from the nearest already-chosen center, yielding provable approximation guarantees and faster convergence in practice [39].

Choosing the number of clusters k is itself a problem. Rousseeuw's silhouette coefficient quantifies how well each point fits its assigned cluster relative to the next-nearest cluster:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \tag{4}$$

where $a(i)$ is the mean intra-cluster distance for point i and $b(i)$ is the mean distance from i to all points in the nearest other cluster, with values close to $+1$ indicating tight well separated clusters [40]. Sweeping k and selecting the value that maximises the mean silhouette gives a principled auto-selection rule. Once clusters are formed, a large language model can produce natural-language cluster explanations [41].

2.4 Retrieval-Augmented Feedback

2.4.1 Retrieval-Augmented Generation

Lewis et al. introduced Retrieval-Augmented Generation (RAG) as a way to combine parametric language models with a non-parametric retriever over an external source, allowing the generator to condition its output on relevant evidence retrieved at query time [42]. Additionally, RAG models hallucinate less and generate factually more correct text than BART [42]. Lewis et al. showed that

RAG outperformed parametric models on knowledge intensive tasks, including open-domain QA tasks [42].

In educational contexts, the case for retrieval is even stronger. Kasneci et al. survey the opportunities and challenges of large language models in education, noting that there is a difficulty in distinguishing between real knowledge and convincingly written but unverified model output due to the ability of LLMs to generate human-like text which can make it difficultt for students to distinguish between real knowledge and unverified model output leading students to accept false or misleading information as true. To mitigate this risk, it is recommended we provide education on how information can be evaluated critically and teach students exploration, investigation, verification and corroboration strategies [43]. Retrieval-augmented generation can help mitigate this risk by grounding the model's output in verifiable sources, allowing students to trace the information back to its origin and evaluate its credibility [42, 43].

2.4.2 Dense Retrieval and Approximate Nearest Neighbour Search

Before the retrieval step in RAG, we need a representation that captures semantic similarity between documents and queries. Reimers and Gurevych proposed Sentence-BERT (SBERT), which is a modification of pre-trained BERT that uses siamese and triplet network structures to derive semantically meaningful sentence embeddings that can be compared using cosine similarity [44]. In the paper they show that commonly used approaches to derive sentence embeddings from BERT such as CLS-token outputs or averaging the BERT output embeddings perform worse than averaged GloVe embeddings on standard semantic textual similarity benchmarks. Siamese fine-tuning on natural language inference data substantially improves correlation on STS benchmarks and outperforms sentence-embedding methods such as InferSent and Universal Sentence Encoder [44].

Once documents are embedded, exact nearest neighbour search scales linearly with the number of documents which is unfeasible for large retrieval over large collections. To relax this, Approximate Nearest Neighbours Search (K-ANNS) was proposed, which relaxes the condition of the exact search by allowing a small number of errors in exchange for faster than linear time complexity [45]. Malkov and Yashunin proposed Hierarchical Navigable Small World (HNSW) graphs, which are a fully graph based ANN index that incrementally builds a multi-layer structure of proximity graphs for nested subsets of the stored elements [45]. As queries descend greedily from top to the zero layer, logarithmic complexity is achieved, with the evaluation showing that HNSW outperforms previous open source vector only approaches [45].

Together retrieval-augmented generation, dense sentence embeddings, and approximate nearest neighbour search build the standard pipeline for grounding LLM output.

2.5 Existing Systems

2.5.1 Learning Management System

A learning management system is a software application used for the administration, documentation, tracking, reporting, automation, and delivery of educational courses, training programs, materials or learning and development programs [46]. Learning management systems also constitute the

largest share of the learning systems market; almost all higher education institutions have adopted them [46]. They were designed to identify training and learning gaps using analytical data and reporting. A notable learning management systems is Moodle which is the world's most popular learning management system [46].

A limitation of these systems are that the analytics are typically reviewed after activities, meaning some data may not be as useful after and students that were struggling in computer labs may have not be provided timely assistance. Another limitation of such systems is that they are primarily designed for course level and while this is useful for curriculum design and long-time planning, they are generally not optimised for in lab computer sessions where timely support is critical.

2.5.2 Instructor Facing Dashboards

Learning management systems often incorporate dashboards to track student or user progress, helping teachers better understand knowledge gaps [47]. Dashboards provide insight into student progress through visualisation and scores derived from data generated during teaching and learning activities. As a result, dashboards are at the centre of learning analytics, integrated with learning management systems [47].

Existing learning dashboards, such as Blocks: Progress Bar, report on student progress rather than student struggle at the live lab level [47]. Dashboards like these show user active time, assignment progress, learning progress, marks, and assignment submissions [47–49]. Visual components such as the progress bar aid in the understanding of student progress, removing all the complexity that is going on behind the scenes. Although these are useful, they don't directly present the user with more complex questions compared to others, and students struggle.

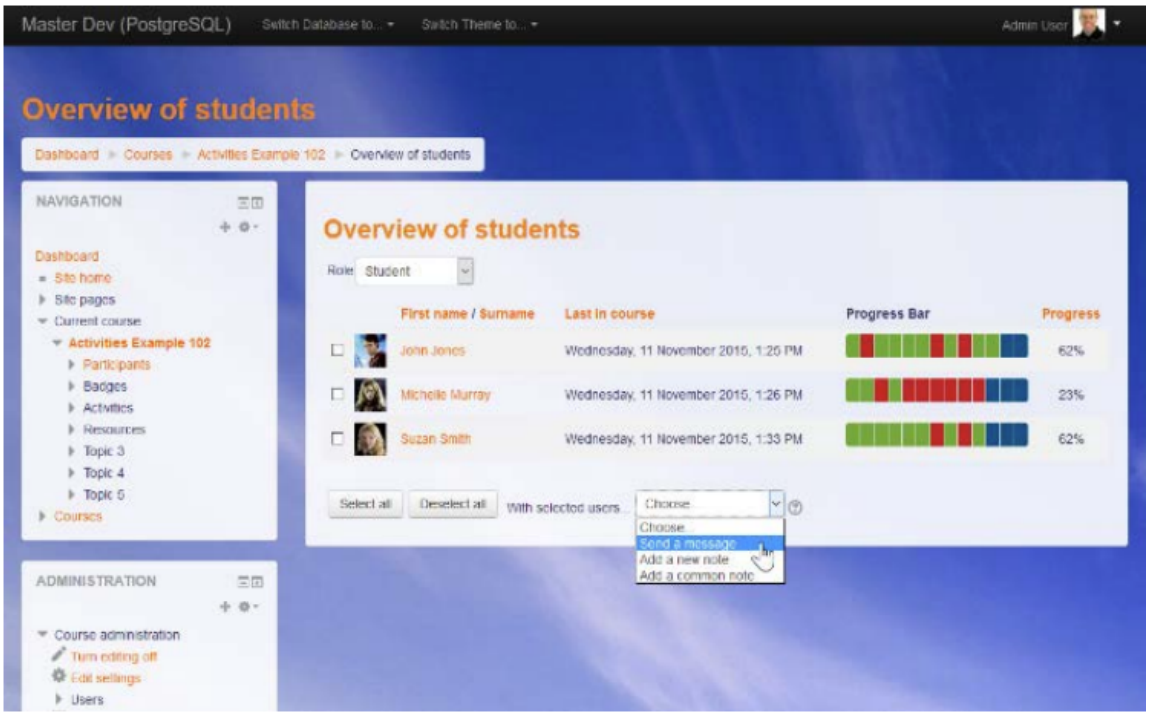

Figure 4: Piwik Analytics

Another implementation of a learning dashboard is the Multi-Modal (MM) teacher dashboard, the MM dashboard: 1) receives data from eye trackers, wristbands, web cameras, and the learning activity; 2) computes the student's relevant cognitive, affective, and physiological measurements; 3) synchronises and visualises the resulting measurements in a clean, customisable interface; and 4) notifies instructors during potential moments of interest [50].

Figure 5: Different views of MM dashboard: SSV (left), ASV (centre), and RSV (right).

The MM dashboard has three ways of data visualisation, as we can see in 5: 1) single session view

(SSV), which displays real-time learning data from a single student's session; 2) all sessions view (ASV), which shows real-time learning data from all connected students; and 3) replay session view (RSV), which allows the teacher to interact with previously recorded session data from a single student session [50]. The views are divided into panels, and we can also see that the SSV and ASV have a right-side navbar used to switch between panels [50].

However, the dashboards do come with a range of issues. For the MM dashboard to work, students must have a range of sensors, which are uncommon for them [50]. Teachers also report no experience with some of the hardware required for the MM dashboard to work. Although this is a key problem, it was not the MM dashboard's main issue; the main issue was that teachers may lack the knowledge to understand and interpret the dashboard results, as the majority of users reported unfamiliarity with the measurements provided. The importance of the metrics was also questioned [50]. The study also noted that hands-on training may be required to help teachers use the system efficiently [50].

The limitations highlight a critical gap in the practicality and potential for utilisation of the learning dashboards. Whilst the dashboards provide high-level insights, many of these insights are difficult to understand, prompting educators to question their importance. Also, these dashboards tend to be unintuitive, which can be especially an issue for educators who are not STEM-oriented. We can leverage these insights to create clearer indicators, such as struggle, and by taking a more straightforward approach to the dashboard, we can address these gaps and create better, more useful learning dashboards.

2.5.3 Early Warning and At-Risk Student Systems

Early warning systems in education identify students at risk of poor learning outcomes by analysing routinely collected data and surface those students to instructors before failure becomes irreversible [51]. In education, an At-risk student system is an early warning system that spots struggling students before it is too late [52]. Recently, many approaches to at-risk student systems have leveraged AI to identify students at risk [52].

Student data is analysed to flag potential issues. Typically, grades, attendance, online activity, and behaviour are used to alert teachers to help students early, before problems grow [52]. The datafication of education enables the use of automated methods to detect patterns in extensive educational data. Approaches leverage predictive models to use the data, predicting the likelihood that a student will fail a course [52].

We can look at the edInsight Early Warning System as an example. This system looks at the course as a whole and uses the information gained to determine an at-risk ranking for all students [53]. The system leverages a scoring system, based on specific categories such as the Course grade category [53]:

Grade	Score
C-	2
D+	4
D	8
F	10

Table 2: Scoring system based on grades, where lower grades add to the score, showing that a student is struggling more

With the scores calculated, educators have a clear idea of which students are most at risk, making this information useful for directing attention to those who need the most help [53]. The same approach has been validated in academic environments. Arnold and Pistilli describe the Course Signals system at Purdue, which is an early intervention system that combines grade, past academic history and demographic predictors into a traffic light indicator for instructors, reporting measurable gains in student retention [54]. Jayaprakash et al. generalises this further with an open-source initiative, the Open Academic Analytics Initiative (OAAI), which has been deployed across multiple US institutions, demonstrating that the underlying predictive techniques are not specific to a single platform [55].

Although the system is beneficial for identifying students who are struggling at the course level, it does not necessarily help identify students at the lab level, as its design assumptions do not align with the demands of lab sessions. Educators working in real-time sessions require live data. Hence, they have immediate awareness of which students are currently struggling, rather than future failure. However, we can apply the same concepts used in at-risk systems to identify students who are struggling the most and questions that have the most difficulty. As a result, as a result while traditional at risk systems are not designed for lab sessions, the underlying principles can be adapted to meet the needs of real-time lab analytics.

2.6 Research Gaps

2.7 Requirements Specification

2.7.1 Functional Requirements

In this section, we define the core functionality required of the proposed system to address the problem in Chapter 1 and the gaps highlighted earlier. These requirements specify what the system must do regardless of implementation.

FR1: Live ingestion of Lab data The system must be capable of ingesting student interaction data as it is generated in the lab sessions, which includes data which is produced whilst students are answering questions, such as attempts amount, how wrong the answer is, how long spent on the question, which will be used for measuring student struggle as well as metrics we will use in measuring question difficulty such as total attempt amount, total correct submissions and average time spent.

FR2: Computation of student struggle The system must compute a metric to represent how much a student is struggling in labs, derived from data highlighted in **FR1**. This representation should represent short-term difficulty rather than long-term ability and predicted outcomes.

FR3: Computation of question difficulty The system must compute a metric for question difficulty used to represent how hard a question or task is in a lab session, derived from the data highlighted in **FR1**. This will represent a task that is difficult for everyone in attendance at the lab.

FR4: Real-time prioritisation The system must provide an indicator of which students are struggling most in computer labs, based on student struggle data, allowing instructors to aid those who need it most.

FR5: Present Relevant Analytics The system must present relevant analytics for the labs in a suitable, intuitive form that can be interpreted quickly to enable quick decision-making.

FR6: Smart Device The system must integrate with smart devices such as phones, watches and glasses, in particular the smartphones so that information can be sent swiftly to lab assistants.

FR7: Lab Assistant Ranking System The system will record and display lab assistant contributions during the session so that assistant activity can be monitored and incentivised.

2.7.2 Non-Functional Requirements

In addition to functional requirements, the system must satisfy non-functional requirements to ensure the system operates as intended. These requirements reflect the practical constraints of the live lab sessions and ensure efficient operation.

NFR1: Real Time Performance The system must process and update analytics with minimal latency to support swift decision-making in lab sessions. Delays should not prevent an instructor from responding to students who need help.

NFR2: Interpretability Outputs should be interpretable by instructors without requiring specialist knowledge or extensive training. The system should be intuitive so that it can be used across all educational disciplines. Analytics should aid in understanding rather than providing overly complex information.

NFR3: Robustness to noisy data The presence of noisy or incomplete data should not interfere with the system's operations. Noisy or incomplete data should be accounted for and not interfere with the provision of correct analytics.

NFR4: Scalability The system should be capable of operating efficiently in various labs regardless of module, class size and question type.

NFR5: Privacy and Ethical Data must be handled responsibly and solely for the purpose of providing analytics in computer labs, and it must not be used for any purpose other than aiding students.

NFR6: Extensibility and Maintainability The system must be designed to accommodate additional data sources or metrics without requiring a fundamental redesign, enabling future extensions.

2.7.3 Requirements Prioritisation (MoSCoW)

For this section, we will use a MosCoW table to highlight the project's requirements prioritisation, reflecting on time constraints and the project scope.

Must Have	Should Have	Could Have	Won't Have
FR1	FR5	FR7	Long term student outcomes
FR2	FR6		Fully AI decision making
FR3	NFR3		Avatar based AI assistance
FR4	NFR4		
NFR1	NFR6		
NFR2			
NFR5			

Table 3: MoSCoW table highlighting requirements for proposed software

3 Design and Modelling

This section describes the design and architecture of the lab dashboard, highlighting how various components will be implemented. Additionally, we will talk about the data endpoint, defining what data enters the system and in what form. We will then discuss the mathematical models we plan to leverage for the project. Finally, we will discuss how we will expose the analytics to educators to provide an intuitive solution.

3.1 System Architecture

The proposed system adopts a layered architecture designed to support real-time learning analytics in computer lab environments. The architecture separates data generation, data ingestion and processing, as well as decision-making and action, into three distinct layers, ensuring each component can evolve independently. Although this focuses on an interval-based approach to the data, an event-driven pipeline is currently under exploration.

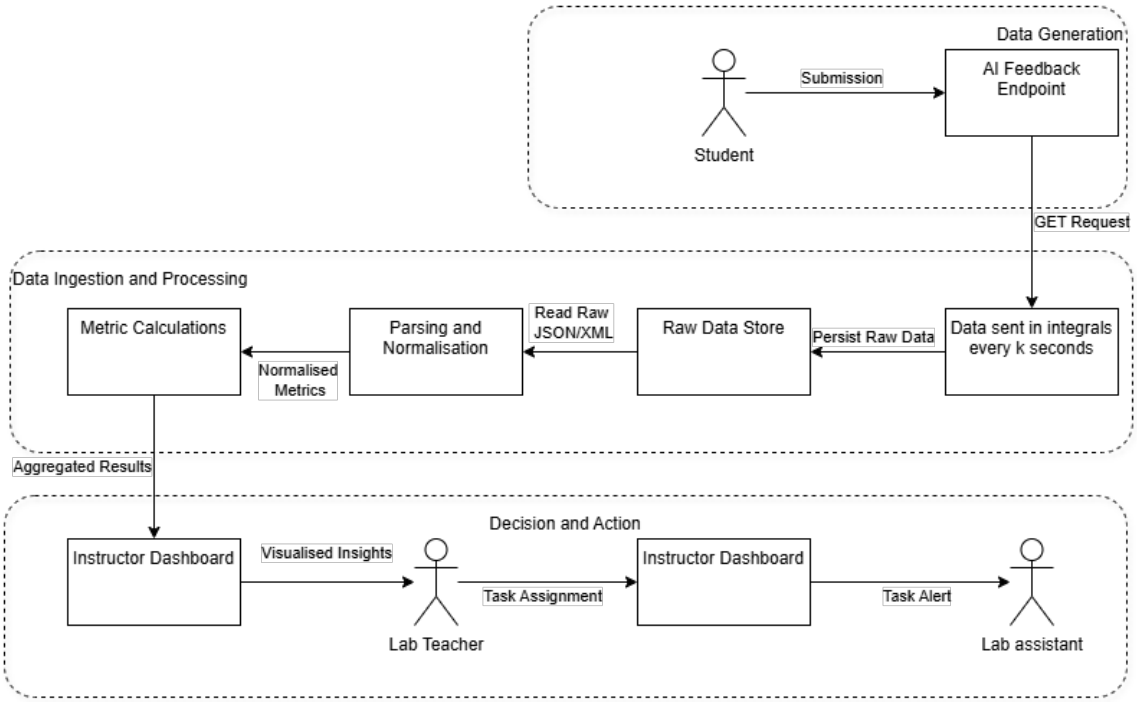

Figure 6: Diagram showing the architecture of the proposed system

Data Generation Layer As seen in Figure 6, student interactions form the primary source of data. When students submit an attempt or request feedback, the interaction data is recorded at the endpoint and not modified.

Data Ingestion and Processing Layer Data ingestion is performed periodically according to the user-defined refresh rate. Retrieved data will be stored in the raw data store, then parsed and normalised before being used for calculations. The data will be used to determine student struggle, question difficulty, and other required metrics.

Decision and Action Layer Data Analytics will now be sent to the learning dashboard for presentation to the instructor, thereby aiding decision-making. The instructor will identify students who are struggling. More complex questions will also be identified so that the instructor can go through the questions with the whole class. Instructors will use the dashboard to assign tasks to assistants to assist students who need help.

This layered design supports the project's aim of providing timely, targeted support during lab sessions while using the system as a decision-making tool for student support. Later sections build on this architecture.

3.2 Data Endpoint

A data endpoint is a specific URL or address in a web service or API that defines where and how clients can access the service. The proposed system integrates with the existing AI feedback platform through a single external data endpoint. The data endpoint is crucial in our system as it provides access to detailed records of student submissions and feedback interactions generated as students engage with lab questions. As outlined in Figure 6, the endpoint serves as the entry point to the analytics pipeline, supplying the raw data that will be processed, normalised, and aggregated for data visualisation. The integration is designed to be read-only, ensuring the data and metrics are not tampered with.

The endpoint returns an array in JSON format, where each record corresponds to a specific session based on the session's first submission time. Alongside standard fields such as module, question, timestamp and user, each record has an XML payload. The payload captures the interaction history for a given session, including submissions and AI responses. The design reflects the constraints of the existing system while providing vital information for our analytics.

Interaction data are organised around sessions, where a session represents a continuous period of student engagement from the first submission attempt. Within a session, students may have multiple submissions, getting multiple AI responses. This is beneficial in reflecting how their answers may improve question by question. The structure allows the capture of temporal patterns, repeated attempts, and help-seeking behaviour, which will be vital for identifying short-term struggle and difficult questions in lab sessions.

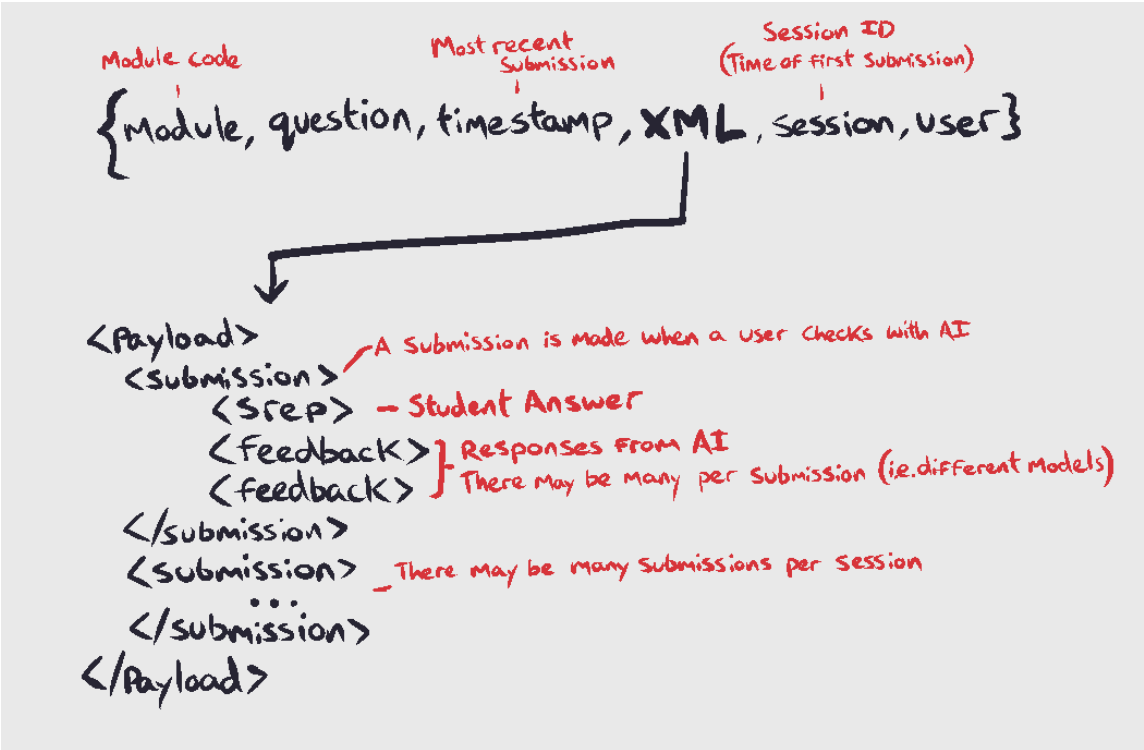

Figure 7: Structure of session-based interaction data returned by feedback system

From the session-based data, the system derives a set of core metrics for real-time monitoring. These include attempts per question, time between attempts, AI-based indicators of correctness, and the frequency with which students request feedback. By using these, we will enable the system to distinguish among students and identify those who are struggling the most. The derived metrics will form the basis of our two key metrics: student struggle and question difficulty.

It is essential to consider that we may receive incomplete or noisy data during sessions, which could stem from students repeatedly returning to the AI feedback system or from procrastination on their work. Not all indicators of struggle and question difficulty are valuable on their own; hence, they are combined to produce indicators of student struggle and question difficulty.

3.3 Baseline Analytics Design

In this section, we will describe what we will measure and what available data will aid in measuring these.

3.3.1 Student Struggle Model

Modelling Objective To model student struggle, let s denote a student and σ denote the session. We will denote students who struggle with a scalar function:

$$S(s, \sigma) \in [0, 1] \tag{5}$$

The student struggle function will allow us to quantify the degree of difficulty that students s are currently experiencing during session σ, based on the observable data from the endpoint in Section 3.2. The model will support real-time prioritisation in computer labs.

Identified Variables From the session-based interaction data described in Section 3.2. We can derive the following quantities:

- $n_{s,\sigma}$ - The total number of submissions made by a student s during session σ
- $t_{s,\sigma}$ - The total active time elapsed during session σ
- $e_{s,\sigma}$ - The number of incorrect submissions with session σ
- $f_{s,\sigma}$ - The number of AI feedback requests within the session σ

By leveraging metrics linked to student struggle, we can produce a student struggle score

AI-Based Answer Rating Let $A_{s,\sigma,k}$ be the k-th submitted answer by student s in session σ. We will derive an LLM-based scoring function. Where Q_σ is the current question and A_σ is the answer, we will define our LLM-based scoring function to be:

$$c_{s,\sigma,k} = C(A_{s,\sigma,k}, A_\sigma, Q_\sigma) \in [0, 1] \tag{6}$$

where $c_{s,\sigma,k}$ is the correctness measure that measures the quality of a students answer, where 1 is the correct answer and 0 is completely wrong. We can then define our incorrectness measure:

$$i_{s,\sigma,k} = 1 - c_{s,\sigma,k} \in [0, 1] \tag{7}$$

Normalisation We will define maximum and minimum amounts as such:

$$n_{min} = \min_{s \in S} n_{s,\sigma}, \quad n_{max} = \max_{s \in S} n_{s,\sigma}$$

We can apply the same definitions to other metrics where relevant. To ensure we can compare the metrics, it is necessary to normalise our quantities:

$$\hat{n_{s,q}} = \frac{n_{s,\sigma} - n_{min}}{n_{max} - n_{min}}, \quad \hat{t_{s,q}} = \frac{t_{s,\sigma} - t_{min}}{t_{max} - t_{min}}$$
$$\hat{e_{s,q}} = \frac{e_{s,\sigma}}{n_{s,\sigma}}, \quad \hat{f_{s,q}} = \frac{f_{s,\sigma}}{n_{s,\sigma}}$$

Hence,

$$\hat{n_{s,\sigma}}, \hat{t_{s,\sigma}}, \hat{e_{s,\sigma}}, \hat{f_{s,\sigma}} \in [0, 1]$$

Give recent submissions more importance To reflect that recent submissions are the most relevant thing in the live setting, we will aggregate the most recent m submissions using convex weights w_i:

$$A^{raw}(s,\sigma) = \sum_{i=0}^{m-1} w_i i_{s,\sigma,k-i}, \quad w_i \geqslant 0, \sum_{i=0}^{m-1} w_i = 1 \tag{8}$$

Since $i_{s,\sigma,k-i} \in [0,1]$, it follows that:

$$A^{raw}(s,\sigma) \in [0,1]$$

Struggle Score Definition Define the raw struggle score as a convex combination of the defined bounded components:

$$\mathbf{S}^{\text{raw}}(s,\sigma) = \alpha \hat{n}_{s,\sigma} + \beta \hat{t}_{s,\sigma} + \gamma \hat{e}_{s,\sigma} + \delta \hat{f}_{s,\sigma} + \eta A^{\text{raw}}(s,\sigma), \tag{9}$$

where:

$$\alpha, \beta, \gamma, \delta, \eta \geqslant 0, \quad \alpha + \beta + \gamma + \delta + \eta = 1$$

Since we are using normalised values it follows that:

$$S^{raw}(s,\sigma) \in [0,1]$$

Temporal updating for real-time operation To provide a responsive, stable indicator, we will update struggle with exponential smoothing:

$$S_t(s,\sigma) = (1-\lambda) S_{t-1}(s,\sigma) + \lambda S^{raw}(s,\sigma), \quad \lambda \in (0,1] \tag{10}$$

It follows that the update preserves boundaries, giving:

$$S_t(s,\sigma) \in [0,1], \quad \forall t$$

Interpretation of model Higher values of $S_t(s,\sigma)$ indicate that student s is experiencing sustained difficulty in the current session. The metric in Equation 3.3.1 will prove to be valuable in real-time prioritisation of help in lab sessions. This measure does not indicate student ability in the cohort but rather in lab struggle.

3.3.2 Temporal Smoothing

3.3.3 Question Difficulty Model

Modelling Objective To model question difficulty, let q denote a question during the lab session, and let $\mathcal{S}_q$ denote the set of student-session pairs associates with task q. We will denote questions that are difficult with a scalar function:

$$D(q) \in [0,1] \tag{11}$$

The question difficulty function will allow us to quantify the degree of difficulty of a question q, that the students in the $\mathcal{S}_q$ set are finding the hardest, based on the observable data from the endpoint in Section 3.2. The model will support real-time prioritisation in computer labs

Identified Variables From the session based interaction data described in Section 3.2. We can derive the following quantities:

- n_q - The total number of attempts on question q
- c_q - The total number of correct attempts on question q
- t_q - The average time spent on question q
- a_q - The average number of attempts spent on question q
- f_q - Average AI incorrectness measure for q, derived from AI-Based answer ratings

By leveraging metrics linked to question difficulty, we can produce a question difficulty score.

Normalisation To ensure we can compare the metrics, it is necessary to normalise our quantities:

$$\tilde{c_q} = 1 - \frac{c_q}{n_q}$$
$$\tilde{t_q} = \frac{t_q - t_{min}}{t_{max} - t_{min}} \quad \tilde{a_q} = \frac{a_q - a_{min}}{a_{max} - a_{min}}$$

f_q is already bounded so:

$$\tilde{f_q} = f_q \in [0, 1]$$

Difficult Task Definition Define question difficulty as a convex combination of the bounded components:

$$D^{\text{raw}}(q) = \alpha\tilde{c}_q + \beta\tilde{t}_q + \gamma\tilde{a}_q + \delta\tilde{f}_q \tag{12}$$

where:

$$\alpha, \beta, \gamma, \delta \geqslant 0, \quad \alpha + \beta + \gamma + \delta = 1$$

Since we are using normalised values it follows that:

$$D^{raw}(q) \in [0, 1] \tag{13}$$

Temporal updating for real-time operation To provide a responsive stable indicator, we will update question difficulty with exponential smoothing:

$$D_t(q) = (1 - \lambda)D_{t-1}(q) + \lambda D^{raw}(q), \quad \lambda \in (0, 1) \tag{14}$$

Interpretation of Model Higher values of $D_t(q)$ indicate that a question is difficult for the class as a whole during the current session. The metric will prove to be valuable in real-time prioritisation of help in lab sessions.

3.3.4 Alternative Modelling Approach: Collaborative Filtering

Motivation The parameter-based struggle model defined in Section 3.3.1 assigns a score to each student independently base on a weighted combination of normalised interaction features. Whilst this approach is computationally lightweight and interpretable, it depends on the weightings $\alpha, \beta, \gamma, \delta, \eta$. We can use an alternative modelling approach based on Collaborative Filtering, where rather than scoring each student in isolation, we explore similarity between students.

Student Interaction Matrix Let $\mathcal{S} = \{s_1, s_2, \ldots, s_n\}$ denotes the set of students present in sessions σ. For each student s_i, we define a five-dimensional feature vector derived from the normalised interaction quantities introduced in Section 3.3.1:

$$\mathbf{x}_{s_i} = \left(\hat{n}_{s_i,\sigma},\ \hat{t}_{s_i,\sigma},\ \hat{e}_{s_i,\sigma},\ \hat{f}_{s_i,\sigma},\ A^{\text{raw}}(s_i,\sigma)\right)^\top \in [0,1]^5 \tag{15}$$

Collecting these vectors row-wise, we define the student interaction matrix:

$$\mathbf{X} \in [0,1]^{n\times 5}, \quad \mathbf{X}_{ij} = [\mathbf{x}_{s_i}]_j \tag{16}$$

Each row of $\mathbf{X}$ encodes one student's current behavioural profile during the session. The matrix is updated at each refresh interval as new data arrives from the endpoint.

Student Similarity Measure To quantify how similar two students are, we apply cosine similarity over their feature vectors. Cosine similarity is preferred over Euclidean distance because it measures the angle between two vectors rather than their absolute magnitudes, making it robust to differences in overall activity levels between students.

$$\text{sim}(s_i, s_j) = \frac{\mathbf{x}_{s_i}^\top \mathbf{x}_{s_j}}{\|\mathbf{x}_{s_i}\|_2 \cdot \|\mathbf{x}_{s_j}\|_2} \in [0,1] \tag{17}$$

Since all components of $\mathbf{x}_{s_i}$ are normalised to $[0,1]$, the dot product is non-negative, and cosine similarity is bounded to $[0,1]$. This yields a symmetric similarity matrix $\mathbf{W} \in [0,1]^{n\times x}$ where $W_{ij} = \text{sim}(s_i, s_j)$ and $W_{ii} = 1$ by definition. A value close to 1 indicates near identical behavioural profiles, whilst a value close to 0 indicates students with very different profiles.

Help-Seeking as a Proxy for Struggle In traditional systems, user ratings serve as the signal across similar users. In the lab setting, no explicit ratings are available, so we define a binary help seeking indicator derived from the parametrix model as a proxy:

$$h_{s,\sigma} = \begin{cases} 1 & \text{if } S_t(s,\sigma) \geq \tau \\ 0 & \text{otherwise} \end{cases} \tag{18}$$

where $\tau \in 0, 1$ is a threshold above which a student is considered to be observably struggling based on their score. This allows collaborative filtering to leverage signals from students who are struggling with similar peers who may not have crossed the threshold, enabling earlier intervention.

Predicted Help Need with k-Nearest Neighbours For a student s for whom $h_{s,\sigma}$, we estimate their latent help need by aggregating the help-seeking behaviour of their k most similar peers. Let $\mathcal{N}_k(s)$ denotes the set of k students with the highest similarity to s, excluding s:

$$\mathcal{N}_k(s) = \underset{s' \in \mathcal{S} \setminus \{s\},\ |\mathcal{N}|=k}{\arg\text{top-}k} \ \text{sim}(s, s') \tag{19}$$

The predicted help-need score is then defined as a similarity-weighted average of the help indicators of the k nearest neighbours.

$$\hat{h}_{s,\sigma} = \frac{\sum_{s' \in \mathcal{N}_k(s)} \text{sim}(s, s') \cdot h_{s',\sigma}}{\sum_{s' \in \mathcal{N}_k(s)} \text{sim}(s, s')} \in [0, 1] \tag{20}$$

A high value of $\hat{h}_{s,\sigma}$ indicates that student s is similar to peers who required help, suggesting they may benefit from intervention.

Comparison with Parameter-Based Model Table 4 summarises the key structural differences between the adopted parametric model and the collaborative filtering alternatively, highlighting the properties that informed the choice of primary approach.

Property	**Parameter-Based Model**	**Collaborative Filtering**
Scoring basis	Weighted formula	Peer similarity
Interpretability	High	Moderate
Cold start	Immediate	Requires $\geq k+1$ active students
Computational complexity	$O(n)$ per update	$O(n^2)$ per update

Table 4: Structural comparison of the parametric struggle model and the collaborative filtering alternative

Justification for Current Approach The parameter-based model is adopted as the primary approach for several reasons. This approach does not require a minimum class size to be considered viable, making it robust to sessions with few students where the similarity matrix $\mathbf{W}$ would be unreliable. Additionally, it produces more reliable results immediately, whereas a collaborative filtering approach results are not reliable at the start.

Nevertheless, the collaborative filtering approach is still going to be implemented alongside the parameter-based approach as a secondary indicator, where like the Bloomberg Terminal, we will allow users to toggle whether they would like this feature on or off.

3.3.5 Mistake Clustering

3.4 Advanced Model Design

3.4.1 Measurement Confidence

3.4.2 Item Response Theory

3.4.3 Bayesian Knowledge Training

3.4.4 Improved Struggle Model

3.5 Retrieval-Augmented Generation Feedback Design

3.6 Interaction and UI Designs

This section describes how the output of our models defined in Section 3.3.1 and 3.3.3 is presented to instructors in a way that enables quick dec'ision-making during lab sessions. The design is intended to provide an intuitive view.

3.6.1 Design Principles

The user interface is driven by the requirement to expose the model outputs defined in Sections 3.3.1 and 3.3.3 in a form that enables rapid, intuitive interpretation and decision-making. Unlike the previous dashboard we explored, this one will be more intuitive and will be used primarily for computer labs.

The primary objective is to present student struggle and question difficulty as distinct signals. Student struggle indicators are derived from the struggle score $S_t(s, \sigma)$ and question difficulty indicators are derived from the difficulty score $D_t(q)$. These two indicators will prove to be valuable in helping educators improve learning outcomes.

A further objective is to allocate lab assistants to students so they can help promptly. This will send a notification to the lab assistant allocating them to students. Another is avoiding exposure of raw data, which has been mitigated by data normalisation.

Finally, the system focuses on supporting decision-making rather than explanatory analysis. The dashboard is intended to guide instructors towards timely intervention by highlighting students and tasks requiring attention and enabling efficient allocation of lab assistants.

3.6.2 Instructor Views

The dashboard provides a real-time overview of student struggles and question difficulty during the computer labs. The dashboard is will allow the instructor to make quick decisions and aid those in need. To illustrate the proposed interface, a dashboard mock-up was created using Figma. The mock-up represents a conceptual design rather than a fully implemented system.

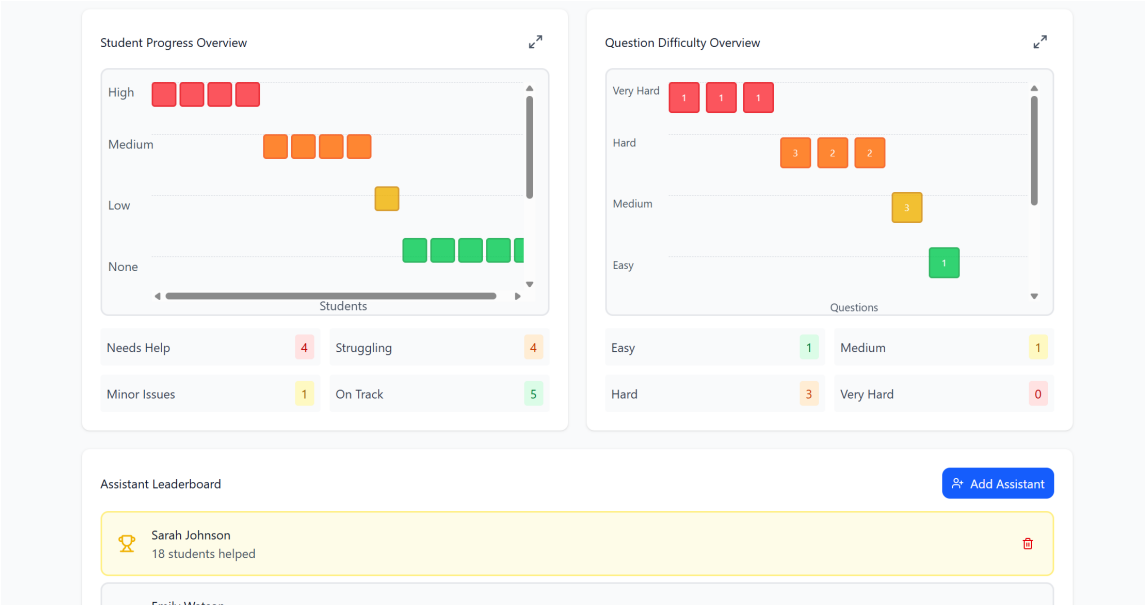

Figure 8: Views based off student struggle and question difficulty

As shown in Figure 8, our dashboard has two key views: the first shows a list of students whose struggles are at certain thresholds, hence the colour; the second shows a list of questions based on difficulty at a threshold, thus the colour. The values used for comparison against the thresholds are derived from our student struggle and question difficulty equations. The view also allows instructors to go over additional information.

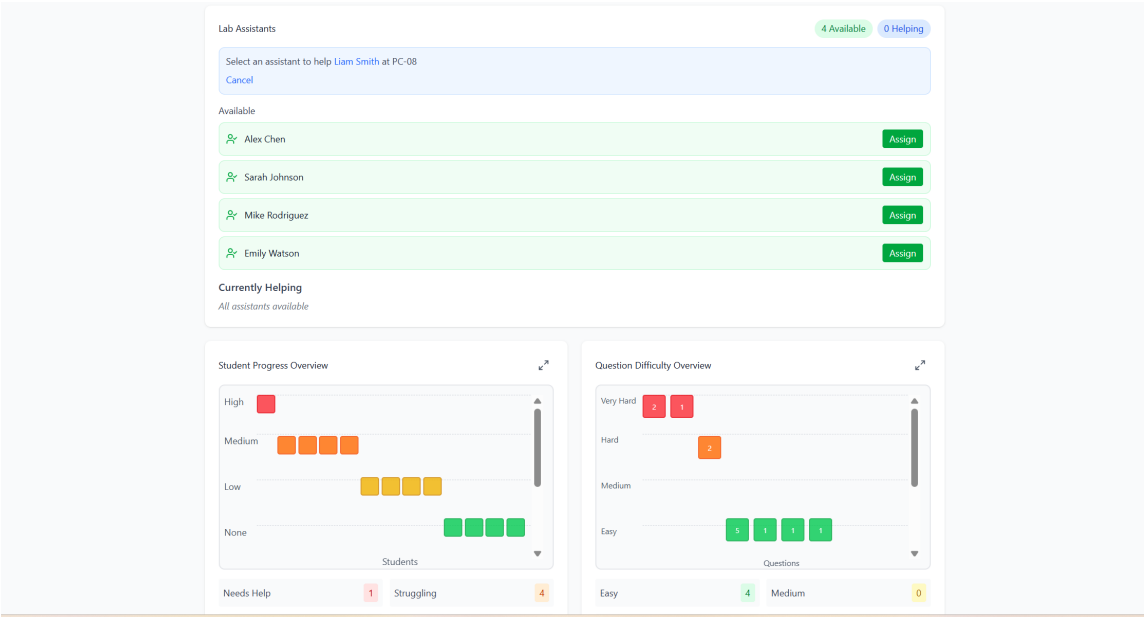

Figure 9: View of allocation of Lab Assistant

In Figure 9, we can select a lab assistant and then press a student to allocate them. This is a simple approach and a very minimal, intuitive design that allows use without training on the system.

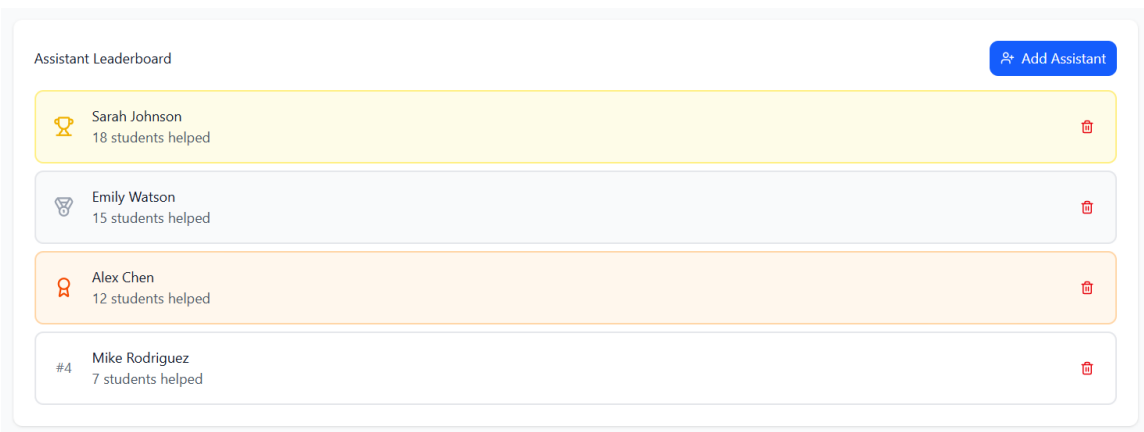

Figure 10: Assistant leaderboard, based on how many students have been help

In Figure 10, we can see that lab assistants are displayed based on their contributions. By gamifying the helping process, we can incentivise assistants to help more students.

3.6.3 Lab Assistant View

3.6.4 Visual Encoding of Metrics

By deciding on thresholds, we can highlight a student by degree of struggle or highlight a question by degree of difficulty

Degree	None	Low	Medium	High
Threshold	$0 \leqslant S_t(s,\sigma) \leqslant t_{s1}$	$t_{s1} < S_t(s,\sigma) \leqslant t_{s2}$	$t_{s2} < S_t(s,\sigma) \leqslant t_{s3}$	$t_{s3} < S_t(s,\sigma) \leqslant 1$
Colour	Green	Yellow	Orange	Red

Table 5: Table Highlighting how thresholds can be indicated for struggle

Degree	Easy	Medium	Hard	Very Hard
Threshold	$0 \leqslant D_t(q) \leqslant t_{s1}$	$t_{s1} < D_t(q) \leqslant t_{s2}$	$t_{s2} < D_t(q) \leqslant t_{s3}$	$t_{s3} < D_t(q) \leqslant 1$
Colour	Green	Yellow	Orange	Red

Table 6: Table Highlighting how thresholds can be indicated for difficulty

4 Implementation

This chapter describes the implemented version of the real-time lab support system. Earlier work focused on validating the endpoint, testing the data pipeline, and establishing a working dashboard with basic dashboard prototype. The current implementation Version 2 (v2) builds on this foundation, allowing for a live instructor dashboard, a lab assistant application that works with the instructor dashboard, session history as well as both baseline and more advanced models.

The implementation is interval-based rather than event-based. However, it provides a full end-to-end workflow, from endpoint ingestion, parsing, normalisation to live scoring, student prioritisation and assistant coordination. The system represents a functional tool to support instructors and assistants during labs allowing for improved learning outcomes, with the potential for further refinement and expansion in future iterations.

4.1 Implementation Overview

The implementation represents Version 1 of the dashboard. Version 1 was implemented to ensure end-to-end functionality worked as planned and to experiment with data visualisation rather than to deploy the whole system proposed.

Version 1 focused on establishing a working data pipeline capable of processing live interaction data during labs, aggregating student activity at the lab level, and presenting indicators in near real time. This provides the foundation for the proposed system. By working on Version 1, I have familiarised myself with the relevant technology and the data pipeline.

The implemented system addresses the core objectives of the project by supporting live monitoring of student progress, prioritisation of students who are struggling, identifications of difficult questions so that lab teachers can go through those questions as a class, and allocation of lab assistants during sessions.

The implementation is made up of a number of components essential to the system's functionality. First

4.2 Technology Stack and Software Selection

System Component	Technology Used	Purpose and Justification
Data Source	PHP-based HTTP endpoint	Existing lab infrastructure exposes student interaction as an endpoint
Data Format	JSON with embedded XML	JSON allows lightweight transport, whilst XML stores structured session-level submission
Data Processing	Python	Provides strong support for parsing of JSON data and performing numerical computations
Analytics Logic	Python	Version 1 has simple indicators for student struggle based on thresholds
Dashboard Framework	Streamlit	Enables fast development of interactive dashboards
Visualisation library	Plotly	Provides interactive visualisations which are intuitive
Deployment Environment	Personal Computer	Suitable for testing and prototyping

Table 7: What was used for each technical aspect and how

The stack highlighted in Table **??** enables a unified development workflow, in which data ingestion, processing and visualisation are implemented within a single Python environment. Whilst Version 1 relies on refreshing, the architecture will be largely consistent with the proposed system.

4.3 System Structure

4.3.1 Instructor System

4.3.2 Assistant System

4.3.3 Shared Runtime State

4.4 Data Pipeline

Version 1 implements an interval-based data pipeline designed to support near real-time monitoring during lab sessions. The pipeline proceeds as follows:

1. **Data Retrieval:** At fixed intervals the Python application re-requests the full dataset from the existing PHP endpoint. This refresh is implemented within the Python code. The refresh helps ensure that metrics are up to date.

2. **Data Parsing:** The retrieved response, formatted as JSON with embedded XML strings, is parsed within Python. Relevant fields such as question, student and timestamps are extracted
3. **Data Structuring:** Parsed records are transformed into an immediate representation suitable for analysis
4. **Metric Computation:** A naive metric approach was used, where thresholds are used for the attempt amount of a question. This was done to demonstrate the concept
5. **Dashboard Update:** The dashboard is refreshed using the newly computed values, providing an updated view of lab activity each time the Python refresh executes

More advanced modelling described in the Design Chapter is not yet integrated and is reserved for the proposed system.

4.4.1 Endpoint Retrieval and Parsing

4.4.2 Data Normalisation and Structuring

4.5 Session Management

4.5.1 Live Session Lifecycle

4.5.2 Saved Session History and Restoration

4.6 Analytics Implementation

4.6.1 Incorrectness Scoring

4.6.2 Baseline Student Struggle Model

4.6.3 Baseline Question Difficulty Model

4.6.4 Collaborative Filtering

4.6.5 Mistake Clustering

4.7 RAG Suggested Feedback

4.7.1 Hybrid Two-Layer Architecture

4.7.2 Embedding Model (all-MiniLM-L6-v2)

4.7.3 Generation and Prompting

4.7.4 Suggestion Caching

4.7.5 Graceful Degradation and Privacy

4.8 Advanced Model Implementation

4.8.1 Measurement Confidence

4.8.2 Item Response Theory (IRT) Difficulty

4.8.3 Bayesian Knowledge Training (BKT) Mastery

4.8.4 Improved Struggle Model

4.9 Lab Instructor System

4.9.1 In Class View

4.9.2 Student Detail View

4.9.3 Question Detail View

4.9.4 Data Analysis View

4.9.5 Comparison View

4.9.6 Settings View

5 Evaluation

5.1 Evaluation Design

5.1.1 Scope of Evaluation

5.1.2 Evidence Sources

5.1.3 Retrospective Evaluation Protocol

5.1.4 Limitations of Evaluation

5.2 Functional Testing

5.2.1 Data Ingestion and Session Handling

5.2.2 Analytics and Model Behaviour

5.2.3 Instructor Dashboard Features

5.2.4 Lab Assistant Workflow

5.3 Non-Functional Testing

5.3.1 Performance and Refresh Behaviour

5.3.2 Usability and Interpretability

5.3.3 Robustness and Failure Handling

5.4 Results

5.4.1 Baseline System Results

5.4.2 Advanced Model Comparison

5.4.3 Observed Strengths and Weaknesses

5.5 Discussion

5.5.1 Model Disagreement

5.5.2 Limitations

6 Conclusion

6.1 Summary of the Project

6.2 Key Findings Contributions

6.3 Future Work

A Code Snippets

B User Interface Screenshots

C Detailed Test Results

D Themes and References

Theme	References
Learning Analytics in Higher Education	[1, 2, 5, 7–9]
Dashboards for Learning Analytics	[10–13]
Real-Time Data Analytics and Visualisation	[14–16]
Modelling Student Struggle	[17–24, 29]
Modelling Question Difficulty	[20, 29, 32, 33]
Collaborative Filtering	[56–59]
Learning Management System	[46]
Instructor Facing Dashboards	[47, 50]
Early Warning and At Risk Student Systems	[51–55]

Table 8: Table highlighted the themes covered and the references used in them

E Formulae

F Formulae Derivation

References

[1] A. Nguyen, L. Gardner, and D. Sheridan, "Data analytics in higher education: An integrated view," *Journal of Information Systems Education*, vol. 31, pp. 61–71, 2020.

[2] W. Contributors, "Learning analytics," May 2019.

[3] G. Siemens, "Learning Analytics: The Emergence of a Discipline," *American Behavioral Scientist*, vol. 57, pp. 1380–1400, Oct. 2013.

[4] D. Clow, "An overview of learning analytics," *Teaching in Higher Education*, vol. 18, pp. 683–695, Aug. 2013.

[5] C. Vieira, P. Parsons, and V. Byrd, "Visual learning analytics of educational data: A systematic literature review and research agenda," *Computers & Education*, vol. 122, pp. 119–135, July 2018.

[6] C. Romero and S. Ventura, "Educational data mining and learning analytics: An updated survey," *WIREs Data Mining and Knowledge Discovery*, vol. 10, p. e1355, May 2020.

[7] A. Gelan, G. Fastré, M. Verjans, N. Martin, G. Janssenswillen, M. Creemers, J. Lieben, B. Depaire, and M. Thomas, "Affordances and limitations of learning analytics for computer-assisted language learning: A case study of the VITAL project," *Computer Assisted Language Learning*, vol. 31, pp. 294–319, Jan. 2018.

[8] A. Wilson, C. Watson, T. L. Thompson, V. Drew, and S. Doyle, "Learning analytics: Challenges and limitations," *Teaching in Higher Education*, vol. 22, pp. 991–1007, May 2017.

[9] D. Tzimas and S. Demetriadis, "Ethical issues in learning analytics: A review of the field," *Educational Technology Research and Development*, vol. 69, pp. 1101–1133, Mar. 2021.

[10] J. Klerkx, K. Verbert, and E. Duval, "Learning analytics dashboards," *Handbook of Learning Analytics*, pp. 143–150, May 2017.

[11] K. Verbert, X. Ochoa, R. De Croon, R. A. Dourado, and T. De Laet, "Learning analytics dashboards," *Proceedings of the Tenth International Conference on Learning Analytics & Knowledge*, 2013.

[12] S. Govaerts, K. Leuven, E. Duval, and K. Verbert, "The student activity meter for awareness and self-reflection," May 2012.

[13] J. Ahn, F. Campos, M. Hays, and D. Digiacomo, "The journal of learning analytics works under a creative commons license," vol. 6, pp. 70–85, 2019.

[14] B. Marr, "What is real-time data analytics (and why it's so important)?," July 2021.

[15] Mustafa Ayobami Raji, Hameedat Bukola Olodo, Timothy Tolulope Oke, Wilhelmina Afua Addy, Onyeka Chrisanctus Ofodile, and Adedoyin Tolulope Oyewole, "Real-time data analytics in retail: A review of USA and global practices," *GSC Advanced Research and Reviews*, vol. 18, pp. 059–065, Mar. 2024.

[16] M. Montebello, "Assisting education through real-time learner analytics," 2018.

[17] Y. Dong, S. Marwan, P. Shabrina, T. Price, and T. Barnes, "Using student trace logs to determine meaningful progress and struggle during programming problem solving."

[18] P. Or, "Exploring novice programmers' testing behavior: A first step to define coding struggle citation in BibTeX format exploring novice programmer testing behavior: A first step to define coding struggle," 2024.

[19] A. Estey, H. Keuning, and Y. Coady, "Automatically classifying students in need of support by detecting changes in programming behaviour," Mar. 2017.

[20] C. Piech, M. Sahami, D. Koller, S. Cooper, and P. Blikstein, "Modeling how students learn to program."

[21] C. Koutcheme, N. Dainese, and A. Hellas, "Using program repair as a proxy for language models' feedback ability in programming education."

[22] G. Pitts, A. Hridi, and A.-B. Lekshmi-Narayanan, "A survey of LLM-based applications in programming education: Balancing automation and human oversight."

[23] M. Khajah, "Supercharging BKT with multidimensional generalizable IRT and skill discovery."

[24] D. Kim, S. Kim, and Y. Jo, "Knowledge tracing in programming education integrating students' questions."

[25] A. T. Corbett and J. R. Anderson, "Knowledge tracing: Modeling the acquisition of procedural knowledge," *User Modelling and User-Adapted Interaction*, vol. 4, no. 4, pp. 253–278, 1995.

[26] M. V. Yudelson, K. R. Koedinger, and G. J. Gordon, "Individualized Bayesian Knowledge Tracing Models," in *Artificial Intelligence in Education* (D. Hutchison, T. Kanade, J. Kittler, J. M. Kleinberg, F. Mattern, J. C. Mitchell, M. Naor, O. Nierstrasz, C. Pandu Rangan, B. Steffen, M. Sudan, D. Terzopoulos, D. Tygar, M. Y. Vardi, G. Weikum, H. C. Lane, K. Yacef, J. Mostow, and P. Pavlik, eds.), vol. 7926, pp. 171–180, Berlin, Heidelberg: Springer Berlin Heidelberg, 2013.

[27] C. Piech, J. Spencer, J. Huang, S. Ganguli, M. Sahami, L. Guibas, and J. Sohl-Dickstein, "Deep Knowledge Tracing," 2015.

[28] M. Khajah, R. V. Lindsey, and M. C. Mozer, "How deep is knowledge tracing?," 2016.

[29] J. Dannath, A. Deriyeva, and B. Paaßen, "Evaluating task-level struggle detection methods in intelligent tutoring systems for programming,"

[30] G. Rasch, *Probabilistic Models for Some Intelligence and Attainment Tests.* Studies in Mathematical Psychology, University of Chicago Press, 1960.

[31] F. Lord and M. Novick, *Statistical Theories of Mental Test Scores.* Addison-Wesley Series in Behavioral Sciences, Emerald Publishing Limited, 1968.

[32] F. Baucks, R. Schmucker, and L. Wiskott, "Gaining insights into course difficulty variations using item response theory," Mar. 2024.

[33] M. Pankiewicz, "Measuring task difficulty for online learning environments where multiple attempts are allowed -the Elo rating algorithm approach."

[34] P. Resnick, N. Iacovou, M. Suchak, P. Bergstrom, and J. Riedl, "GroupLens: An open architecture for collaborative filtering of netnews," in *Proceedings of the 1994 ACM Conference on Computer Supported Cooperative Work - CSCW '94*, (Chapel Hill, North Carolina, United States), pp. 175–186, ACM Press, 1994.

[35] J. L. Herlocker, J. A. Konstan, A. Borchers, and J. Riedl, "An algorithmic framework for performing collaborative filtering," in *Proceedings of the 22nd Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, (Berkeley California USA), pp. 230–237, ACM, Aug. 1999.

[36] G. Salton, A. Wong, and C. S. Yang, "A vector space model for automatic indexing," *Communications of the ACM*, vol. 18, pp. 613–620, Nov. 1975.

[37] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 1 ed., 2006.

[38] J. MacQueen, "Some methods for classification and analysis of multivariate observations," 1967.

[39] D. Arthur and S. Vassilvitskii, "K-means++: The Advantages of Careful Seeding," 2007.

[40] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *Journal of Computational and Applied Mathematics*, vol. 20, pp. 53–65, Nov. 1987.

[41] Z. Wang, J. Shang, and R. Zhong, "Goal-Driven Explainable Clustering via Language Descriptions," 2023.

[42] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," 2020.

[43] E. Kasneci, K. Sessler, S. Küchemann, M. Bannert, D. Dementieva, F. Fischer, U. Gasser, G. Groh, S. Günnemann, E. Hüllermeier, S. Krusche, G. Kutyniok, T. Michaeli, C. Nerdel, J. Pfeffer, O. Poquet, M. Sailer, A. Schmidt, T. Seidel, M. Stadler, J. Weller, J. Kuhn, and G. Kasneci, "ChatGPT for good? On opportunities and challenges of large language models for education," *Learning and Individual Differences*, vol. 103, p. 102274, Apr. 2023.

[44] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," 2019.

[45] Yu. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs," 2016.

[46] W. Contributors, "Learning management system," Sept. 2019.

[47] O. Kiat Xin and D. Singh, "Development of learning analytics dashboard based on moodle learning management system," *IJACSA) International Journal of Advanced Computer Science and Applications*, vol. 12, p. 2021, Jan. 2021.

[48] K. Holstein, B. M. McLaren, and V. Aleven, "Intelligent tutors as teachers' aides: Exploring teacher needs for real-time analytics in blended classrooms," in *Proceedings of the Seventh International Learning Analytics & Knowledge Conference*, (Vancouver British Columbia Canada), pp. 257–266, ACM, Mar. 2017.

[49] K. Holstein, B. M. McLaren, and V. Aleven, "Student Learning Benefits of a Mixed-Reality Teacher Awareness Tool in AI-Enhanced Classrooms," in *Artificial Intelligence in Education* (C. Penstein Rosé, R. Martínez-Maldonado, H. U. Hoppe, R. Luckin, M. Mavrikis, K. Porayska-Pomsta, B. McLaren, and B. Du Boulay, eds.), vol. 10947, pp. 154–168, Cham: Springer International Publishing, 2018.

[50] S. Lee-Cultura, K. Sharma, and M. N. Giannakos, "MultiModal teacher dashboards: Challenges and opportunities of enhancing teacher insights through a case study," pp. 1–19, Jan. 2023.

[51] L. P. Macfadyen and S. Dawson, "Mining LMS data to develop an "early warning system" for educators: A proof of concept," *Computers & Education*, vol. 54, pp. 588–599, Feb. 2010.

[52] D. Bañeres, M. E. Rodríguez, A. E. Guerrero-Roldán, and A. Karadeniz, "An early warning system to detect at-risk students in online higher education," *Applied Sciences*, vol. 10, p. 4427, June 2020.

[53] edInsight, "At-risk students early reporting system - edInsight," Sept. 2020.

[54] K. E. Arnold and M. D. Pistilli, "Course signals at Purdue: Using learning analytics to increase student success," in *Proceedings of the 2nd International Conference on Learning Analytics and Knowledge*, LAK '12, (New York, NY, USA), pp. 267–270, Association for Computing Machinery, Apr. 2012.

[55] S. M. Jayaprakash, E. W. Moody, E. J. M. Lauría, J. R. Regan, and J. D. Baron, "Early Alert of Academically At-Risk Students: An Open Source Analytics Initiative," *Journal of Learning Analytics*, vol. 1, pp. 6–47, May 2014.

[56] J. B. Schafer, D. Frankowski, J. Herlocker, and S. Sen, "Collaborative filtering recommender systems," *The Adaptive Web*, vol. 4321, pp. 291–324, 2007.

[57] F. L. da Silva, B. K. Slodkowski, K. K. A. da Silva, and S. C. Cazella, "A systematic literature review on educational recommender systems for teaching and learning: Research trends, limitations and opportunities," *Education and Information Technologies*, vol. 28, pp. 3289–3328, Sept. 2022.

[58] M. Deschênes, "Recommender systems to support learners' Agency in a Learning Context: A systematic review," *International Journal of Educational Technology in Higher Education*, vol. 17, Oct. 2020.

[59] J. Li and Z. Ye, "Course recommendations in online education based on collaborative filtering recommendation algorithm," *Complexity*, vol. 2020, p. e6619249, Dec. 2020.